

La programmation orientée objet

Fil conducteur. On reprend le système du document 1, le gestionnaire d'ordres de trading. Au document 1, un ordre était un simple dictionnaire : `{"symbole": "ES", "quantite": 5, ...}`. Ici, on va le transformer en un véritable objet, une classe `Ordre`, qui regroupe à la fois les données (symbole, quantité, prix) et les actions qu'on peut faire dessus (valider, exécuter). C'est tout l'intérêt de la programmation orientée objet : réunir au même endroit ce qu'une chose est et ce qu'elle sait faire.

Note sur le terme « orienté objet ». La programmation orientée objet n'est pas un langage : c'est un style de programmation (un « paradigme ») que de nombreux langages permettent — C++, C#, Java, et Python aussi. Python est multi-paradigme : on peut y écrire sans objets (style procédural, comme le dict du document 1) ou avec des classes (ce document). Les deux sont du Python valide.

Comment lire ce document. Chaque section présente d'abord l'idée en mots simples, puis un exemple commenté, puis une explication. Le texte entre les blocs de code porte l'essentiel de la compréhension : ne le saute pas.

Les quatre piliers de la POO

Avant d'entrer dans le code, voici la carte mentale du sujet. La programmation orientée objet repose traditionnellement sur **quatre piliers**. Ce document les illustre tous ; garde-les en tête, ils structurent tout ce qui suit.

1. **Encapsulation** — regrouper les données et leur logique au même endroit, et contrôler l'accès aux données (sections 2, 3 et 7).
2. **Héritage** — placer le commun dans une classe parente, le spécifique dans des classes enfants (section 4).
3. **Polymorphisme** — un même appel de méthode, des comportements différents selon le type réel de l'objet (section 5).
4. **Abstraction** — exposer l'essentiel (ce qu'un objet sait faire) en cachant les détails ; on peut même imposer un contrat commun via une classe abstraite (section 5.2).

On les croiera dans cet ordre, appliqués au gestionnaire d'ordres.

1. Classe et objet

classe — Une **classe** est un plan de construction : elle décrit ce que sont les objets d'un certain type. Un **objet** (ou « instance ») est un exemplaire concret fabriqué à partir de ce plan. On construira progressivement cette hiérarchie :

```
Ordre
├── OrdreLimite
└── OrdreMarket
```

Tous les ordres possèdent : `symbole`, `quantite`, `prix`

1.1 Première version de la classe Ordre

```
class Ordre:
    def __init__(self, symbole, quantite, prix):
        self.symbole = symbole
        self.quantite = quantite
        self.prix = prix

# Création de deux objets (deux instances)
o1 = Ordre("ES", 5, 5234.50)
o2 = Ordre("NQ", 2, 18450.00)

o1.symbole    # -> "ES"    (on accède à un attribut avec un point)
o2.quantite    # -> 2
```

1.2 Comprendre __init__

Lorsque Python exécute `o1 = Ordre("ES", 5, 5234.50)`, il effectue trois opérations : (1) **création** d'un nouvel objet, (2) **initialisation** de cet objet, (3) retour de l'objet dans `o1`. Ces deux premières étapes correspondent à deux méthodes distinctes, qu'il ne faut pas confondre :

Méthode	Rôle
<code>__new__</code>	Le constructeur au sens strict : il crée l'objet (alloue l'instance) et la renvoie. Rarement redéfini en pratique.
<code>__init__</code>	L' initialiseur : il ne crée pas l'objet, il le remplit — il reçoit l'instance déjà créée et lui affecte ses attributs. C'est lui qu'on écrit presque toujours.

Autrement dit : `__new__` fabrique l'objet, puis `__init__` l'initialise. Dans la pratique courante on ne touche qu'à `__init__`, mais il est important de savoir que ce *n'est pas* lui le véritable constructeur. Par abus de langage, on entend souvent appeler `__init__` « le constructeur » ; c'est une approximation commode mais techniquement inexacte.

1.3 Comprendre self

`self` représente l'objet courant. Les affectations `self.symbole = symbole` signifient « dans CET objet précis, range la valeur reçue dans l'attribut `symbole` ». Chaque objet possède ensuite ses propres valeurs, indépendantes des autres. On ne passe jamais `self` explicitement à l'appel : Python le fournit automatiquement, `o1.methode()` revenant à `methode(o1)`.

1.4 Attribut d'instance contre attribut de classe

Tous les attributs vus jusqu'ici (préfixés de `self.`) sont des **attributs d'instance** : chaque objet a les siens. Il existe aussi des **attributs de classe**, déclarés directement dans le corps de la classe et **partagés par toutes les instances**. Exemple typique : un compteur d'ordres créés.

```
class Ordre:
    compteur = 0                                # attribut de CLASSE (partagé)

    def __init__(self, symbole, quantite, prix):
        self.symbole = symbole                  # attribut d'INSTANCE (propre à chaque objet)
        self.quantite = quantite
```

```

        self.prix = prix
        Ordre.compteur += 1          # on incrémente le compteur partagé

a = Ordre("ES", 5, 5234.50)
b = Ordre("NQ", 2, 18450.00)
Ordre.compteur      # -> 2   (commun aux deux objets)
a.symbole           # -> "ES" (propre à 'a')

```

Règle simple : ce qui caractérise **un objet en particulier** (son symbole, sa quantité) est un attribut d'instance (`self.x`) ; ce qui est commun à **toute la classe** (un compteur, une constante, une configuration) est un attribut de classe. Les débutants confondent souvent les deux — la section suivante montre le piège le plus dangereux.

1.5 Le piège des attributs de classe mutables

Un attribut de classe **mutable** (liste, dictionnaire) est partagé par toutes les instances — ce qui crée un bug classique :

```

class Ordre:
    fills = []          # PIÈGE : liste partagée par TOUTES les
instances

    def ajouter_fill(self, prix, qte):
        self.fills.append((prix, qte)) # modifie la liste commune

a = Ordre()
b = Ordre()
a.ajouter_fill(5234.25, 4)
b.fills      # -> [(5234.25, 4)]  !! le fill de 'a' apparaît aussi chez 'b'

```

La solution est de créer la liste **dans** `__init__`, pour que chaque objet ait la sienne :

```

class Ordre:
    def __init__(self):
        self.fills = []      # une liste PROPRE à chaque instance

a = Ordre(); b = Ordre()
a.fills.append((5234.25, 4))
b.fills      # -> []   (correct : indépendant de 'a')

```

À retenir : les valeurs **immuables** (int, str, float) en attribut de classe ne posent pas problème ; les valeurs **mutables** (list, dict, set) doivent presque toujours être initialisées dans `__init__`. On reverra ce piège avec les dataclasses (section 8.3).

Pourquoi ça compte. Réunir les données d'un ordre dans un objet structuré évite les dictionnaires aux clés incertaines (a-t-on écrit "quantite" ou "quantity" ?). Le code devient plus sûr et plus lisible, ce qui compte quand des ordres réels engagent de l'argent.

2. Les méthodes

Une **méthode** est une fonction définie dans une classe. Elle décrit ce que l'objet sait faire et accède à ses attributs via `self`.

```

class Ordre:

```

```

def __init__(self, symbole, quantite, prix):
    self.symbole = symbole
    self.quantite = quantite
    self.prix = prix

def valeur_notionnelle(self):
    return self.quantite * self.prix

```

```

ordre = Ordre("ES", 5, 5234.50)
ordre.valeur_notionnelle()    # -> 26172.50

```

La méthode reçoit `self` en premier, ce qui lui donne accès aux attributs : `valeur_notionnelle` lit `self.quantite` et `self.prix` pour les multiplier. À l'appel `ordre.valeur_notionnelle()`, Python fournit `ordre` comme `self` automatiquement.

2.1 Trois sortes de méthodes : normale, de classe, statique

Il existe en réalité trois types de méthodes, qui se distinguent par **ce qu'elles reçoivent en premier argument** :

Type	Reçoit	Usage
Méthode normale	<code>self</code>	L'instance courante. Le cas par défaut.
<code>@classmethod</code>	<code>cls</code>	La classe elle-même (pas l'instance). Sert surtout aux constructeurs alternatifs.
<code>@staticmethod</code>	rien	Ni <code>self</code> ni <code>cls</code> . Une simple fonction logée dans la classe pour la cohérence.

La méthode de classe (`@classmethod`). Très fréquente pour les **constructeurs alternatifs** : une fabrique qui sait construire l'objet à partir d'une autre forme de données. Elle reçoit `cls` (la classe) et renvoie une nouvelle instance.

```

class Ordre:
    def __init__(self, symbole, quantite, prix):
        self.symbole = symbole
        self.quantite = quantite
        self.prix = prix

    @classmethod
    def depuis_dict(cls, d):
        # constructeur alternatif
        return cls(d["symbole"], d["quantite"], d["prix"])

d = {"symbole": "ES", "quantite": 5, "prix": 5234.50}
o = Ordre.depuis_dict(d)    # construit un Ordre à partir d'un dict

```

Pourquoi `cls(...)` et non `Ordre(...)` ? Parce qu'avec `cls`, si une sous-classe (`OrdreLimite`) appelle `depuis_dict`, la fabrique construit automatiquement un `OrdreLimite`, pas un `Ordre`. Le constructeur alternatif suit l'héritage.

La méthode statique (`@staticmethod`). Elle n'a besoin *ni* de l'instance *ni* de la classe : c'est un utilitaire rangé dans la classe par cohérence thématique.

```

class Ordre:
    @staticmethod

```

```
def tick_valide(prix, pas=0.25):
    # vérifie que le prix tombe sur un pas de cotation
    return round(prix / pas) * pas == prix

Ordre.tick_valide(5234.50)    # -> True  (appelée sans instance)
Ordre.tick_valide(5234.30)    # -> False
```

2.2 Méthode ou property ?

La valeur notionnelle est-elle une **action** ou une **caractéristique** ? C'est une caractéristique. On peut donc la modéliser par une **property** (section 7), utilisée sans parenthèses. Le critère « caractéristique vs action » est un guide utile, mais **pas absolu** : une property doit rester **peu coûteuse**, car on l'appelle comme un simple attribut. Si le calcul devient lourd (requête réseau, parcours d'une grande structure), beaucoup de développeurs préfèrent en faire une méthode explicite `ordre.calculer_valeur()` — les parenthèses signalant alors « ceci a un coût ». Pour une validation pouvant devenir onéreuse, écrire `ordre.est_valide()` (méthode) est tout à fait défendable, au lieu de `ordre.est_valide` (property).

***Pourquoi ça compte.** Comparé au document 1 où il fallait une fonction externe `valider(o)`, la logique est désormais attachée à l'objet : moins d'erreurs, code plus cohérent.*

3. Encapsulation

L'**encapsulation** consiste à contrôler l'accès aux données d'un objet pour empêcher qu'on les corrompe. C'est une notion qu'on construit en deux temps : d'abord une convention (le tiret bas), puis un vrai contrôle (la property avec setter). Les deux sont présentés ici, de bout en bout.

3.1 Le problème

```
ordre = Ordre("ES", 5, 5234.50)
ordre.quantite = -500    # Python l'autorise... mais ça n'a aucun sens
```

Rien n'empêche d'affecter une quantité négative, qui n'a aucun sens métier. On veut interdire cela.

3.2 Premier temps : le tiret bas (une convention)

```
class Ordre:
    def __init__(self, symbole, quantite, prix):
        self.symbole = symbole
        self._quantite = quantite    # le tiret bas : "attribut interne"
        self.prix = prix
```

Le tiret bas devant `_quantite` signale « attribut interne, ne pas modifier directement de l'extérieur ». C'est une **convention** que tous les développeurs respectent : en lisant `ordre._quantite`, on comprend qu'on touche à quelque chose d'interne. Mais attention, le tiret bas **ne protège rien techniquement** :

`ordre._quantite = -500` fonctionne toujours, Python ne l'interdit pas. Le tiret bas sert seulement à stocker la valeur réelle ; il prépare le terrain pour la véritable encapsulation, ci-dessous.

3.3 Second temps : la property avec setter (le vrai contrôle)

Pour réellement **contrôler** l'écriture, on combine le tiret bas avec une **property** et un **setter**. La property `quantite` devient l'interface publique, et le setter valide toute affectation avant de stocker la valeur dans `_quantite`.

```
class Ordre:
    def __init__(self, symbole, quantite, prix):
        self.symbole = symbole
        self.quantite = quantite          # passe déjà par le setter ci-dessous
        self.prix = prix

    @property
    def quantite(self):                  # le "getter" : lecture
        return self._quantite

    @quantite.setter
    def quantite(self, valeur):          # le "setter" : écriture contrôlée
        if valeur <= 0:
            raise ValueError("La quantité doit être positive")
        self._quantite = valeur

ordre = Ordre("ES", 5, 5234.50)
ordre.quantite = 10                    # OK, passe par la validation
ordre.quantite = -500                  # -> ValueError : la quantité doit être positive
```

Désormais `ordre.quantite = -500` déclenche le setter, qui rejette la valeur. La valeur réelle est stockée dans `_quantite`, et toute lecture ou écriture passe par la property. C'est cela, la véritable encapsulation : non plus une simple convention, mais une barrière active.

3.4 Le piège de la récursion infinie

Un point crucial, souvent source de bug : dans le setter, il faut stocker dans `_quantite` (avec tiret bas), **pas** dans `quantite`. Écrire `self.quantite = valeur` dans le setter rappellerait le setter lui-même, encore et encore : c'est une **récursion infinie** (erreur `RecursionError`). Le tiret bas n'est donc pas décoratif : il distingue l'attribut de stockage (interne) de la property (interface).

***Pourquoi ça compte.** Sur un compte ou un ordre de trading, garantir qu'une valeur ne peut jamais devenir négative ou nulle, où qu'on la modifie dans le code, évite des bugs coûteux. La property centralise cette règle en un seul endroit.*

4. Héritage

Dans un système de trading, tous les ordres ne se comportent pas pareil. Considérons deux types : l'**ordre limite** (prix fixé à l'avance) et l'**ordre market** (exécuté immédiatement au meilleur prix). Les deux partagent `symbole`, `quantite`, `cote`, mais diffèrent sur le reste. L'**héritage** permet de placer le commun dans une classe parente et le spécifique dans des classes enfants, sans duplication.

4.1 La classe parente

```
class Ordre:
    def __init__(self, symbole, quantite, cote):
        self.symbole = symbole
```

```
self.quantite = quantite
self.cote = cote
```

Cette classe contient uniquement ce qui est commun à tous les ordres.

4.2 OrdreLimite : ajouter un prix limite

```
class OrdreLimite(Ordre):
    def __init__(self, symbole, quantite, cote, prix_limite):
        super().__init__(symbole, quantite, cote)  # réutilise le parent
        self.prix_limite = prix_limite            # attribut spécifique
```

La classe `OrdreLimite(Ordre)` hérite de `Ordre`. `super()` réutilise l'initialisation du parent au lieu de recopier les trois affectations, puis on ajoute `prix_limite`.

Que désigne vraiment `super()` ? Tant qu'on n'a qu'un seul parent, on peut lire `super()` comme « ma classe parente ». Mais ce n'est pas exactement cela : `super()` suit l'**ordre de résolution des méthodes** (le **MRO**, *Method Resolution Order*), c'est-à-dire l'ordre dans lequel Python parcourt les classes pour trouver une méthode. En héritage simple, ce MRO se réduit au parent direct, donc la lecture intuitive suffit. Mais en **héritage multiple** (une classe avec plusieurs parents), `super()` peut pointer vers une classe « sœur » et non vers le parent direct. Retiens donc « `super()` suit le MRO » plutôt que « `super()` appelle mon parent », pour ne pas être surpris plus tard.

4.3 OrdreMarket : une liste d'exécutions

```
class OrdreMarket(Ordre):
    def __init__(self, symbole, quantite, cote):
        super().__init__(symbole, quantite, cote)
        self.fills = []  # les exécutions réelles, ajoutées au fil du temps

om = OrdreMarket("ES", 10, "ACHAT")
om.fills  # -> [] (aucune exécution encore)
om.fills.append((5234.25, 6))
om.fills.append((5234.50, 4))  # 6 + 4 = 10 : la quantité demandée est remplie
```

Un ordre market accumule ses exécutions (les « fills ») dans une liste. Ici, deux fills de 6 et 4 contrats remplissent exactement les 10 contrats demandés. Chaque classe enfant a son propre `__init__` parce qu'elle ajoute un attribut spécifique, mais toutes réutilisent le parent via `super()` pour éviter de recopier le code commun.

Pourquoi ça compte. L'héritage évite la duplication : la logique commune est écrite une fois dans le parent. Pour ajouter un type d'ordre (stop, iceberg...), on dérive simplement, sans toucher au reste.

4.4 Héritage ou composition ? « est un » contre « possède un »

L'héritage est puissant, mais ce n'est **pas** la réponse à tout. Avant de dériver une classe, pose-toi la question de la relation entre les deux objets :

Relation	Exemple
« est un » → héritage	Un <code>OrdreLimite</code> est un <code>Ordre</code> . La relation est de spécialisation : on hérite.

Relation	Exemple
« possède un » → composition	Un Compte <i>possède des</i> ordres. Le compte n'est pas un ordre ; il en <i>contient</i> . On l'exprime par un attribut, pas par l'héritage.

La **composition** consiste à donner à un objet d'autres objets comme attributs :

```
class Compte:
    def __init__(self, identifiant):
        self.identifiant = identifiant
        self.ordres = []                # le compte POSSÈDE des ordres

    def ajouter(self, ordre):
        self.ordres.append(ordre)

compte = Compte("ACC-001")
compte.ajouter(OrdreLimite("ES", 5, "ACHAT", 5234.50))
len(compte.ordres)    # -> 1
```

Erreur fréquente du débutant : faire *hériter* Compte de Ordre parce que les deux « sont liés ». C'est faux — un compte n'est pas une sorte d'ordre. La règle pratique : si tu peux dire « X **est un** Y », hérite ; si tu dis « X **possède un** Y », compose. En conception objet, on privilégie souvent la composition, plus souple que l'héritage.

5. Le polymorphisme

L'héritage seul n'apporte pas grand-chose ; sa vraie valeur apparaît avec le **polymorphisme** : des objets de classes différentes répondent au même appel de méthode, chacun à sa façon. Donnons à chaque classe une méthode `description()`. Pour que l'exemple soit autonome, on redonne ici les trois classes **complètes** (constructeurs inclus).

```
class Ordre:
    def __init__(self, symbole, quantite, cote):
        self.symbole = symbole
        self.quantite = quantite
        self.cote = cote

    def description(self):
        # texte lisible décrivant l'ordre
        return f"{self.cote} {self.quantite} {self.symbole}" # version par défaut

class OrdreLimite(Ordre):
    def __init__(self, symbole, quantite, cote, prix_limite):
        super().__init__(symbole, quantite, cote)
        self.prix_limite = prix_limite

    def description(self):
        return f"{self.cote} {self.quantite} {self.symbole} @ {self.prix_limite}"

class OrdreMarket(Ordre):
    def __init__(self, symbole, quantite, cote):
        super().__init__(symbole, quantite, cote)
        self.fills = []
```



```
def description(self):
    return f"{self.cote} {self.quantite} {self.symbole} ({len(self.fills)} fills)"
```

La méthode `description()` a un rôle métier simple : produire une **ligne lisible** décrivant l'ordre (pour un journal, un écran de suivi). Chaque type d'ordre la formate à sa manière. Mettons-la à l'épreuve :

```
ol = OrdreLimite("ES", 5, "ACHAT", 5234.50)
om = OrdreMarket("NQ", 10, "VENTE")
om.fills.append((18450.0, 6))
om.fills.append((18451.0, 4))      # 2 fills enregistrés

ol.description()    # -> "ACHAT 5 ES @ 5234.50"
om.description()    # -> "VENTE 10 NQ (2 fills)"
```

Même nom de méthode, comportement différent : c'est le polymorphisme. (Note la cohérence : `om` a reçu deux fills, et `description()` affiche bien « 2 fills ».) Son intérêt se révèle sur une liste mixte d'ordres.

5.1 L'intérêt concret

Avant la boucle, un mot sur `isinstance(objet, Classe)` : cette fonction intégrée renvoie `True` si `objet` est une instance de `Classe` (ou d'une de ses sous-classes). Elle sert à tester le type réel d'un objet — précisément ce que le polymorphisme va nous permettre d'éviter :

```
ordres = [OrdreLimite("ES", 5, "ACHAT", 5234.50), OrdreMarket("NQ", 10, "VENTE")]

# Sans polymorphisme : on teste le type de chaque objet avec isinstance (lourd)
for ordre in ordres:
    if isinstance(ordre, OrdreLimite):
        print(ordre.description())
    elif isinstance(ordre, OrdreMarket):
        print(ordre.description())

# Avec polymorphisme : un seul appel, Python choisit la bonne version
for ordre in ordres:
    print(ordre.description())
```

La boucle ne connaît pas le type réel de chaque objet : elle sait seulement que chacun possède une méthode `description()`. Python choisit automatiquement la bonne version à l'exécution. Comme une télécommande universelle où le bouton « Power » allume l'appareil présent, quel qu'il soit. Cela rend le code plus simple et extensible : ajouter un type d'ordre ne change pas la boucle.

5.2 Imposer un contrat : les classes abstraites (ABC)

Le polymorphisme suppose que *chaque* sous-classe fournit bien une méthode `description()`. Mais rien ne l'y oblige : si on oublie de l'écrire dans une nouvelle sous-classe, le bug ne se révèle qu'à l'exécution. Une **classe abstraite** rend ce contrat explicite et **le fait respecter**. C'est ici qu'intervient le quatrième pilier, l'**abstraction**.

```
from abc import ABC, abstractmethod

class Ordre(ABC):
    # ABC = classe de base abstraite
    def __init__(self, symbole, quantite, cote):
        self.symbole = symbole
        self.quantite = quantite
        self.cote = cote
```

```

@abstractmethod
def description(self):          # contrat : toute sous-classe DOIT la fournir
    ...

class OrdreLimite(Ordre):
    def __init__(self, symbole, quantite, cote, prix_limite):
        super().__init__(symbole, quantite, cote)
        self.prix_limite = prix_limite

    def description(self):        # le contrat est honoré
        return f"{self.cote} {self.quantite} {self.symbole} @ {self.prix_limite}"

Ordre("ES", 5, "ACHAT")          # -> TypeError : Can't instantiate abstract class Ordre
OrdreLimite("ES", 5, "ACHAT", 5234.50)  # OK : la sous-classe est concrète

```

Deux effets : (1) on ne peut **plus instancier** `Ordre` directement — c'est un plan incomplet ; (2) toute sous-classe qui *oublie* d'implémenter `description()` devient elle-même impossible à instancier, et l'erreur survient **tôt**, pas au fond d'une boucle en production. La classe abstraite documente et **garantit** l'interface commune.

Pourquoi ça compte. Traiter une liste d'ordres variés de façon uniforme, sans cascade de tests de type, rend le code générique et prêt pour de nouveaux types d'ordres. Une classe abstraite garantit en plus que chaque nouveau type respectera le contrat — une sécurité précieuse quand l'équipe s'agrandit.

5.3 Python ne « surcharge » pas les méthodes

Venant de C++, C# ou Java, on s'attend à pouvoir définir **plusieurs** méthodes de même nom mais aux signatures différentes (la *surcharge*, *overloading*). **Ce n'est pas le cas en Python** : si deux méthodes portent le même nom, la seconde **écrase** purement et simplement la première.

```

class Ordre:
    def __init__(self, symbole):          # cette définition...
        self.symbole = symbole

    def __init__(self, symbole, quantite): # ...est ÉCRASÉE par celle-ci
        self.symbole = symbole
        self.quantite = quantite

Ordre("ES")          # -> TypeError : il manque l'argument 'quantite'
                    # seule la dernière __init__ existe

```

En Python, on obtient le même effet autrement : des **arguments par défaut** (`def __init__(self, symbole, quantite=1)`), des arguments variables (`*args, **kwargs`), ou des **constructeurs alternatifs** via `@classmethod` (section 2.1). Ne pas confondre avec la *redéfinition* (*overriding*), bien réelle elle, où une sous-classe redéfinit une méthode du parent — c'est exactement ce que fait `description()` à la section 5.

6. Les méthodes spéciales (dunder)

Les méthodes entourées de doubles tirets bas (comme `__init__`) sont appelées méthodes spéciales, ou « dunder » (pour *double underscore*). Elles permettent à un objet de se comporter comme un type natif : s'afficher, se comparer. On les écrit **dans** la classe, au même titre que les autres méthodes.

```
class Ordre:
    def __init__(self, symbole, quantite, prix):
        self.symbole = symbole
        self.quantite = quantite
        self.prix = prix

    def __repr__(self):
        return f"Ordre({self.symbole!r}, {self.quantite}, {self.prix})"

    def __str__(self):
        return f"{self.quantite} {self.symbole} @ {self.prix}"

    def __eq__(self, autre):
        return (self.symbole == autre.symbole
                and self.quantite == autre.quantite
                and self.prix == autre.prix)
```

6.1 `__repr__` et `__str__` : deux affichages, pas un

On résume souvent — à tort — par « `__repr__` définit l’affichage via `print` ». La réalité est plus précise : Python dispose de **deux** représentations textuelles.

Méthode	Pour qui / quoi
<code>__repr__</code>	Représentation technique, sans ambiguïté , destinée au développeur (débogage, journaux, console). Idéalement, elle ressemble à un appel reconstruisant l’objet : <code>Ordre('ES', 5, 5234.5)</code> .
<code>__str__</code>	Représentation lisible , destinée à l’utilisateur final. Plus courte, plus « humaine » : <code>5 ES @ 5234.5</code> .

Qui appelle quoi ? `print(obj)` et `str(obj)` cherchent d’abord `__str__` ; si elle n’existe pas, ils **retombent** (*fallback*) sur `__repr__`. À l’inverse, la console interactive et `repr(obj)` utilisent toujours `__repr__`.

Conséquence pratique : si tu ne dois écrire **qu’une** des deux, écris `__repr__` — elle sert de filet de sécurité partout.

```
o = Ordre("ES", 5, 5234.50)

print(o)      # -> 5 ES @ 5234.5          (utilise __str__)
str(o)        # -> "5 ES @ 5234.5"        (utilise __str__)
repr(o)       # -> "Ordre('ES', 5, 5234.5)" (utilise __repr__)
o             # -> Ordre('ES', 5, 5234.5) (console : __repr__)

# Sans __str__, print() retombe sur __repr__ :
# print(o) afficherait alors Ordre('ES', 5, 5234.5)
```

Sans aucune des deux, l’affichage donne `<__main__.Ordre object at 0x1234>` (illisible). Définir au moins `__repr__` est donc un réflexe précieux pour le débogage et les journaux.

6.2 `__eq__` : l’égalité par contenu

Par défaut, deux objets ne sont égaux que s’ils sont le **même** objet en mémoire. `__eq__` définit l’égalité par contenu.

```
a = Ordre("ES", 5, 5234.50)
```

```
b = Ordre("ES", 5, 5234.50)
a == b      # -> True   (même contenu, grâce à __eq__)
a is b      # -> False  (objets DISTINCTS en mémoire)
```

Attention à la distinction : `==` compare le **contenu** (via `__eq__`), tandis que `is` compare l'**identité** — si ce sont littéralement le même objet en mémoire. Deux ordres au contenu identique mais créés séparément donnent `a == b` vrai mais `a is b` faux. Sans `__eq__`, `a == b` renverrait aussi False.

6.3 Quelques autres dunder utiles

Dunder	Sens	Déclenché par
<code>__len__</code>	longueur	<code>len(mon_objet)</code>
<code>__lt__</code>	« plus petit que »	tri des objets (<)
<code>__add__</code>	addition	<code>objet1 + objet2</code>
<code>__str__</code>	texte « lisible »	<code>str(mon_objet)</code>

6.4 Définir sa propre exception (rappel du document 1, appliqué aux objets)

On peut créer ses propres types d'erreurs pour exprimer des règles métier. Une **exception personnalisée** est une classe qui hérite de `Exception` — deux lignes suffisent — et se manipule comme les exceptions intégrées.

```
class OrdreInvalideError(Exception):
    pass

def valider(ordre):
    if ordre.quantite <= 0:
        raise OrdreInvalideError("La quantité doit être positive")
    return True

try:
    valider(Ordre("ES", -5, 5234.50))
except OrdreInvalideError as e:
    print(f"Ordre rejeté : {e}")    # -> Ordre rejeté : La quantité doit être positive
```

Hériter de `Exception` suffit (le `pass` signifie « rien à ajouter »). On la déclenche avec `raise`, et le code appelant peut l'attraper spécifiquement avec `except OrdreInvalideError`, en la distinguant des autres erreurs. C'est le pont avec la gestion d'exceptions du document 1, ici au service d'une classe.

Pourquoi ça compte. Les dunder rendent les objets intuitifs (affichage, comparaison), et une exception personnalisée nommée rend le code métier clair : on sait immédiatement, en lisant `OrdreInvalideError`, de quel problème il s'agit.

7. Les property

Une **property** est une méthode qu'on utilise sans parenthèses, comme un attribut. On l'a déjà rencontrée à la section 3 pour valider une écriture ; voici ses deux usages complets.

7.1 Caractéristique ou action ?

La règle de décision : si c'est une **caractéristique** de l'objet, c'est une property (sans parenthèses) ; si c'est une **action**, c'est une méthode (avec parenthèses). Garde toutefois en tête la nuance de la section 2.2 : une property doit rester **peu coûteuse**.

CARACTÉRISTIQUE (property, sans parenthèses)	ACTION (méthode, avec parenthèses)
ordre.valeur	ordre.executer()
compte.solde	ordre.annuler()

```
class Ordre:
    def __init__(self, symbole, quantite, prix):
        self.symbole = symbole
        self.quantite = quantite
        self.prix = prix

    @property
    def valeur(self):
        return self.quantite * self.prix

ordre = Ordre("ES", 5, 5234.50)
ordre.valeur      # -> 26172.50    (sans parenthèses, comme un attribut)
```

On lit « quelle est la valeur de cet ordre ? » comme une donnée, d'où la property. La valeur est recalculée à chaque accès, toujours cohérente avec la quantité et le prix actuels.

7.2 Property en lecture seule

Si une property n'a **pas** de setter, elle est en **lecture seule** : toute tentative d'écriture lève une `AttributeError`.

```
class Compte:
    def __init__(self, solde):
        self._solde = solde

    @property
    def solde(self):
        return self._solde

c = Compte(1000)
c.solde      # -> 1000    (lecture autorisée)
c.solde = 2000 # -> AttributeError : property 'solde' has no setter
```

C'est utile pour exposer une valeur sans permettre de la modifier de l'extérieur. Pour autoriser l'écriture, il faut ajouter un setter (sections 3.3 et 7.3).

7.3 Property avec validation : plusieurs exemples

Le second usage, vu à la section 3, est de **contrôler l'écriture** via un setter. Voici une validation combinant type et valeur.

Validation d'un type et d'une valeur :

```
@property
def prix(self):
    return self._prix
```

```

    @prix.setter
    def prix(self, valeur):
        if not isinstance(valeur, (int, float)):
            raise TypeError("Le prix doit être un nombre")
        if valeur <= 0:
            raise ValueError("Le prix doit être positif")
        self._prix = valeur

```

Ici on valide à la fois le **type** (avec `isinstance`) et la **valeur**. Une property peut enchaîner plusieurs vérifications. On peut aussi normaliser une valeur (par exemple `valeur.strip().upper()` pour un symbole) ou la restreindre à un ensemble autorisé.

Pourquoi ça compte. Centraliser la validation dans une property garantit qu’une règle s’applique partout où l’attribut est modifié. Sur des ordres réels, c’est une barrière directe contre les données corrompues.

8. Les dataclasses

Beaucoup de classes ne font que regrouper des données et répètent le même code : un `__init__` qui recopie les paramètres, un `__repr__`, un `__eq__`. La **dataclass** génère ce code automatiquement.

8.1 Version longue contre version dataclass

```

# Version longue (à la main)
class Ordre:
    def __init__(self, symbole, quantite, prix):
        self.symbole = symbole
        self.quantite = quantite
        self.prix = prix
    def __repr__(self):
        return f"Ordre({self.symbole!r}, {self.quantite}, {self.prix})"
    def __eq__(self, autre):
        return (self.symbole == autre.symbole
                and self.quantite == autre.quantite
                and self.prix == autre.prix)

# Version dataclass (équivalente)
from dataclasses import dataclass

@dataclass
class Ordre:
    symbole: str
    quantite: int
    prix: float

```

Les deux produisent une classe identique à l’usage. `@dataclass` génère `__init__`, `__repr__` et `__eq__` à partir des attributs déclarés. Ce code généré est trivial — mécanique et prévisible — c’est pourquoi on peut l’automatiser sans risque.

```

a = Ordre("ES", 5, 5234.50)
b = Ordre("ES", 5, 5234.50)
print(a)    # -> Ordre(symbole='ES', quantite=5, prix=5234.5)
a == b      # -> True (le __eq__ généré compare le contenu)
a is b      # -> False (objets distincts)

```

8.2 Quand choisir l'une ou l'autre ?

La dataclass est avantageuse quand l'objet sert avant tout à porter des données : un ordre, une transaction, une position. On gagne en concision et on évite les fautes de recopie. La classe ordinaire est préférable quand l'objet est surtout défini par ses comportements (beaucoup de méthodes), quand la construction demande une logique élaborée, ou quand on veut un contrôle fin sur `__init__`, `__repr__` ou `__eq__`.

8.3 Le piège des champs mutables : `field(default_factory=...)`

On a vu (section 1.5) qu'une liste partagée entre instances est un bug. Le piège revient avec les dataclasses : on ne peut **pas** donner directement une liste comme valeur par défaut. Python lève d'ailleurs une erreur explicite.

```
from dataclasses import dataclass

@dataclass
class OrdreMarket:
    symbole: str
    fills: list = []          # -> ValueError : mutable default ... is not allowed
```

La solution dédiée des dataclasses est `field(default_factory=...)` : on fournit non pas une valeur, mais une **fonction** (ici `list`) que Python appelle **pour chaque nouvelle instance**, garantissant une liste neuve à chacune.

```
from dataclasses import dataclass, field

@dataclass
class OrdreMarket:
    symbole: str
    fills: list = field(default_factory=list)  # une liste NEUVE par instance

a = OrdreMarket("ES")
b = OrdreMarket("NQ")
a.fills.append((5234.25, 6))
b.fills          # -> [] (correct : indépendant de 'a')
```

Retiens `field(default_factory=list)` (ou `dict`, `set`...) dès qu'une dataclass doit contenir une collection mutable. C'est l'équivalent, côté dataclass, du « initialiser dans `__init__` » de la section 1.5.

Pourquoi ça compte. Un moteur de trading manipule quantité de petits objets de données (ordres, ticks, positions) : les dataclasses y suppriment énormément de code répétitif. Mais une connexion à un courtier, riche en logique, restera une classe ordinaire — et tout champ « liste de fills » exigera `default_factory`.

9. Les enums

Un **enum** (énumération) définit un ensemble fermé de valeurs nommées. Représenter une notion par une chaîne libre est risqué : `cote = "achatt"` (faute de frappe) ne lève aucune erreur, le bug est silencieux. L'enum supprime ce risque.

9.1 Le côté d'un ordre

```
from enum import Enum
```

```

class Cote(Enum):
    ACHAT = "achat"
    VENTE = "vente"

Cote.ACHAT          # -> Cote.ACHAT
Cote.ACHAT.value    # -> "achat"
Cote.ACHETER        # -> AttributeError : la valeur n'existe pas

```

Une faute comme `Cote.ACHETER` provoque une erreur immédiate, au lieu de passer inaperçue.

9.2 Le statut d'un ordre (un cycle de vie)

```

class Statut(Enum):
    EN_ATTENTE = "en_attente"
    EXECUTE = "execute"
    ANNULE = "annule"

statut = Statut.EN_ATTENTE
if statut == Statut.EXECUTE:
    print("L'ordre est passé")

```

On voit d'un coup les seuls états possibles. Comparer avec `statut == Statut.EXECUTE` est clair et sûr.

9.3 La distinction `.name` et `.value`

```

class TypeOrdre(Enum):
    MARCHE = "market"
    LIMITE = "limite"

t = TypeOrdre.LIMITE
t.name    # -> "LIMITE" (le nom du membre, en majuscules)
t.value   # -> "limite" (la valeur associée)

```

On distingue `.name` (le nom du membre) et `.value` (la valeur attribuée). Cela permet d'envoyer "limite" à une API tout en manipulant `TypeOrdre.LIMITE` dans le code.

Pourquoi ça compte. Côté, statut, type d'ordre : autant de notions à valeurs fixes. Les représenter par des enums élimine une catégorie entière de bugs silencieux dus aux chaînes mal orthographiées.

10. Mise en commun

Voici une version réunissant tous les concepts : `dataclass`, `enum`, `property`, méthode et constructeur alternatif, avec des exemples d'utilisation. On y trouve aussi la conversion d'un dictionnaire (document 1) en objet — le pont entre les deux documents.

```

from dataclasses import dataclass, field
from enum import Enum

class Cote(Enum):
    ACHAT = "achat"
    VENTE = "vente"

@dataclass
class Ordre:

```



```

    symbole: str
    quantite: int
    prix: float
    cote: Cote = Cote.ACHAT          # valeur par défaut (immuable : OK
directement)
    fills: list = field(default_factory=list)  # liste mutable : default_factory

    @property
    def valeur(self):                  # caractéristique calculée
        return self.quantite * self.prix

    def est_valide(self):              # comportement (action -> méthode)
        return self.quantite > 0 and self.prix > 0

    @classmethod
    def depuis_dict(cls, d):           # constructeur alternatif
        return cls(d["symbole"], d["quantite"], d["prix"])

```

10.1 Utilisation

```

o = Ordre("ES", 5, 5234.50, Cote.ACHAT)
o.valeur          # -> 26172.50
o.est_valide()    # -> True
print(o)          # -> Ordre(symbole='ES', quantite=5, prix=5234.5,
                    #          cote=<Cote.ACHAT: 'achat'>, fills=[])

```

10.2 Du dictionnaire à l'objet

Au document 1, un ordre était un dictionnaire. On le convertit en objet `Ordre` via le constructeur alternatif, puis une liste entière par compréhension.

```

dicts = [
    {"symbole": "ES", "quantite": 5, "prix": 5234.50},
    {"symbole": "NQ", "quantite": 2, "prix": 18450.0},
]

objets = [Ordre.depuis_dict(d) for d in dicts]  # compréhension (doc 1) + objets (doc 2)

```

`depuis_dict` lit les clés du dictionnaire et construit l'objet ; la compréhension convertit toute la liste. On voit le lien direct entre les deux documents : le dict du document 1 devient l'objet du document 2.

On obtient un système orienté objet cohérent et extensible. Pour ajouter un ordre stop, il suffirait de dériver une nouvelle classe — ou, si l'on a posé une classe abstraite (section 5.2), d'en fournir une implémentation respectant le contrat.

Récapitulatif

Les quatre piliers : encapsulation, héritage, polymorphisme, abstraction. La carte mentale de tout le document.

1. Classe et objet : la classe est le plan, l'objet l'instance ; `self` désigne l'objet courant. `__new__` crée l'objet, `__init__` l'initialise. Distinguer attribut d'instance (`self.x`) et attribut de classe (partagé) ; ne jamais mettre une collection mutable en attribut de classe.

- 2. Méthodes** : réunissent comportements et données via self. Trois sortes : normale (self), @classmethod (cls, pour les constructeurs alternatifs), @staticmethod (rien).
- 3. Encapsulation** : le tiret bas est une convention ; la property avec setter est le vrai contrôle. Attention à la récursion infinie (stocker dans _quantite, pas quantite).
- 4. Héritage** : super() suit le MRO (pas forcément « le parent direct »). « est un » → héritage ; « possède un » → composition.
- 5. Polymorphisme** : un même appel (description) donne des comportements différents. Une classe abstraite (ABC, @abstractmethod) impose le contrat. Python ne surcharge pas les méthodes (la dernière écrase la première) ; il redéfinit (overriding) via l'héritage.
- 6. Dunder** : __repr__ (technique, fallback universel) et __str__ (lisible, pour print) ; __eq__ (égalité par contenu, à distinguer de is) ; exceptions personnalisées via Exception.
- 7. Property** : une caractéristique peu coûteuse (sans parenthèses) ; lecture seule sans setter (AttributeError), validation avec setter.
- 8. Dataclass** : génère __init__ / __repr__ / __eq__ ; champs mutables via field(default_factory=list).
- 9. Enum** : ensemble fermé de valeurs ; distinguer .name et .value.

Questions de vérification

Essaie de répondre avant de lire la réponse, fournie juste en dessous, en retrait.

Sur les classes, méthodes et self

Q1. Quelle est la différence entre une classe et un objet ?

Réponse. La classe est le plan de construction (le moule) ; l'objet est un exemplaire concret fabriqué à partir de ce plan.

Q2. Quel est le rôle de __init__, et est-ce vraiment « le constructeur » ?

Réponse. __init__ initialise l'objet (lui affecte ses attributs) ; Python l'appelle automatiquement à chaque création. Ce n'est pas, au sens strict, le constructeur : la création réelle de l'objet est faite par __new__, appelée juste avant __init__. On parle souvent de __init__ comme « le constructeur » par abus de langage.

Q3. Que représente self, et pourquoi ne le passe-t-on pas à l'appel ?

Réponse. self est l'objet courant. On ne le passe pas explicitement car Python le fournit automatiquement : o.methode() revient à methode(o).

Q4. Différence entre un attribut d'instance et un attribut de classe ?

Réponse. L'attribut d'instance (self.x) est propre à chaque objet ; l'attribut de classe, déclaré dans le corps de la classe, est partagé par toutes les instances. Une collection mutable (liste, dict) en attribut de classe est un piège : elle est partagée — il faut l'initialiser dans __init__.

Q5. Quand utilise-t-on @classmethod plutôt qu'une méthode normale ?

Réponse. Quand la méthode a besoin de la classe (cls) et non d'une instance — typiquement pour un constructeur alternatif (fabrique) qui renvoie une nouvelle instance, comme depuis_dict. Utiliser cls(...) plutôt que le nom de la classe fait suivre l'héritage.

Sur l'encapsulation

Q6. Que signifie un attribut préfixé d'un tiret bas, et protège-t-il vraiment ?

Réponse. Il signale un attribut interne, à ne pas modifier directement de l'extérieur. C'est une convention : techniquement, Python n'empêche pas de le modifier.

Q7. Comment contrôler réellement l'écriture d'un attribut ?

Réponse. Avec une property et un setter : le setter valide la valeur avant de la stocker (par exemple refuser une quantité négative), et lève une exception si elle est invalide.

Q8. Pourquoi un setter doit-il stocker dans _quantite et non dans quantite ?

Réponse. Parce que stocker dans self.quantite (le nom de la property) rappellerait le setter lui-même indéfiniment : c'est une récursion infinie. Le tiret bas distingue l'attribut de stockage de la property.

Sur l'héritage et le polymorphisme

Q9. Qu'apporte super(), et que désigne-t-il exactement ?

Réponse. Il permet de réutiliser le code d'une classe « au-dessus », par exemple super().__init__() pour ne pas recopier l'initialisation commune. Précisément, super() suit l'ordre de résolution des méthodes (MRO) ; en héritage simple cela revient au parent direct, mais en héritage multiple il peut désigner une autre classe.

Q10. Quand préfère-t-on la composition à l'héritage ?

Réponse. Quand la relation est « possède un » plutôt que « est un ». Un Compte possède des ordres (composition : un attribut liste) ; il n'est pas une sorte d'ordre, donc il n'en hérite pas.

Q11. Qu'est-ce que le polymorphisme ?

Réponse. Des objets de classes différentes répondent au même appel de méthode, chacun à sa façon. On traite une liste d'objets variés de manière uniforme, sans connaître leur type exact (sans cascade de isinstance).

Q12. À quoi sert une classe abstraite (ABC, @abstractmethod) ?

Réponse. À imposer un contrat : on ne peut pas instancier la classe abstraite elle-même, et toute sous-classe qui n'implémente pas les méthodes abstraites devient elle aussi non instanciable. L'erreur survient tôt, pas au fond d'une boucle en production. C'est le pilier « abstraction ».

Q13. Python permet-il la surcharge de méthodes (deux mêmes noms, signatures différentes) ?

Réponse. Non. La seconde définition écrase la première ; seule la dernière existe. On obtient des effets équivalents via les arguments par défaut, *args/**kwargs ou @classmethod. À ne pas confondre avec la redéfinition (overriding) par héritage, elle bien réelle.

Sur les dunder

Q14. Quelle est la différence entre __repr__ et __str__, et laquelle print() utilise-t-il ?

Réponse. `__repr__` est la représentation technique sans ambiguïté (pour le développeur) ; `__str__` est la représentation lisible (pour l'utilisateur). `print()` et `str()` utilisent `__str__` et retombent sur `__repr__` si `__str__` n'existe pas. La console et `repr()` utilisent toujours `__repr__`. Si l'on n'en écrit qu'une, c'est `__repr__`.

Q15. Quelle est la différence entre `==` et `is` ?

Réponse. `==` compare le contenu (via `__eq__`) ; `is` compare l'identité, c'est-à-dire si ce sont littéralement le même objet en mémoire. Deux objets de même contenu créés séparément donnent `==` vrai mais `is` faux.

Q16. Comment crée-t-on sa propre exception, et avec quel mot-clé la déclenche-t-on ?

Réponse. On définit une classe qui hérite de `Exception` (`class MonErreur(Exception): pass`). On la déclenche avec `raise` et on peut l'attraper avec `except MonErreur`.

Sur les property

Q17. Comment décider entre une property et une méthode ?

Réponse. Si c'est une caractéristique de l'objet et que son calcul est peu coûteux, c'est une property (sans parenthèses). Si c'est une action, ou si le calcul est coûteux (réseau, gros parcours), c'est une méthode (avec parenthèses) — les parenthèses signalant alors un coût.

Q18. Que se passe-t-il si on affecte une valeur à une property sans setter ?

Réponse. Python lève une `AttributeError` : la property est en lecture seule tant qu'aucun setter n'est défini.

Sur les dataclasses et enums

Q19. Que génère automatiquement une dataclass ?

Réponse. Le constructeur `__init__`, l'affichage `__repr__` et la comparaison `__eq__`, à partir des attributs déclarés.

Q20. Comment donner une liste (ou un dict) par défaut à un champ de dataclass, et pourquoi pas directement ?

Réponse. Avec `field(default_factory=list)`. On ne peut pas écrire `fills: list = []` : Python l'interdit, car la même liste serait partagée par toutes les instances (le piège des défauts mutables). `default_factory` appelle la fonction pour chaque instance, donnant une collection neuve à chacune.

Q21. Quand préférer une classe ordinaire à une dataclass ?

Réponse. Quand l'objet est surtout défini par ses comportements (beaucoup de méthodes), quand la construction demande une logique élaborée, ou quand on veut un contrôle fin sur `__init__`, `__repr__` ou `__eq__`.

Q22. Quel problème un enum résout-il, et quelle est la différence entre `.name` et `.value` ?

Réponse. `.name` est le nom du membre (ex. "LIMITE"), `.value` la valeur attribuée (ex. "limite"). L'enum limite les valeurs à un ensemble fixe et nommé : une faute de frappe provoque une erreur immédiate au lieu d'un bug silencieux.

Cumulatif (documents 1 et 2)

Q23. Comment convertir un dictionnaire d'ordre (doc 1) en objet `Ordre` (doc 2) ?

Réponse. Avec un constructeur alternatif `@classmethod` qui lit les clés du dict et construit l'objet :
`@classmethod def depuis_dict(cls, d): return cls(d["symbole"], d["quantite"], d["prix"])`. On convertit une liste entière par compréhension.

Q24. Quels avantages la classe apporte-t-elle sur le dictionnaire pour représenter un ordre ?

Réponse. Validation des données (property/setters), lisibilité et sûreté d'accès (`o.quantite` vs `o["quantite"]`), regroupement des comportements (méthodes avec les données), et extensibilité (héritage, polymorphisme). Le dict reste préférable pour des données simples ou de structure variable, comme un message JSON brut.

Document suivant (3/8) : les concepts intermédiaires — decorators, context managers (with), générateurs et type hints, toujours appliqués au gestionnaire d'ordres.